

# **EXPERIMENT 1**

## **AIM:**

To implement Linear Search and Binary Search algorithms and analyze their time complexity by comparing their execution time for different input sizes using Virtual Labs.

## **SOFTWARE USED**

- Virtual Labs

## **PROCEDURE**

### Step 1: Select Algorithms

- Open the Virtual Labs simulation for algorithm comparison.
- The simulation is divided into five steps/categories.
- Use the Next Step and Previous Step buttons to navigate between categories.
- Select Step 1: Linear vs Logarithmic.
- Select:
  - Linear Search
  - Binary Search
- Click the + Algorithm Name buttons to add the required algorithms for comparison.

### Step 2: Choose Maximum Input Size (N)

- Select the required maximum input size using the available radio buttons.
- Choose different input sizes to observe how execution time changes.
- The available values of N depend on the selected category.

### Step 3: Set Simulation Time

- Use the Max Simulation Time dropdown.
- The default simulation time is generally 5 seconds.
- Increase the simulation time if required for larger input sizes.

- This prevents computationally expensive algorithms from running for an excessive amount of time.

#### Step 4: Lock and Run

- Check the Lock Values checkbox.
- Click Generate Inputs and Run Algorithm.
- Wait until the selected algorithms finish execution.
- The simulation generates input data and measures the execution time of the algorithms.

#### Step 5: Analyze Results

- Observe the execution-time table.
- Compare the execution times of Linear Search and Binary Search for different values of N.
- Analyze how the execution time changes as N increases.
- Compare the observed results with the theoretical time complexities.

### ALGORITHM

#### Algorithm 1: Linear Search

##### Steps

- Start from the first element of the array.
- Compare the current element with the target.
- If both are equal, return the index of the element.
- Otherwise, move to the next element.
- Continue until the target is found or the array ends.
- If the target is not found, return -1.
- Stop.

##### Pseudocode:

```
LinearSearch(arr, target):  
    for i = 0 to n - 1:  
        if arr[i] == target:
```

```
        return i
    return -1
```

## Algorithm 2: Binary Search

### Steps

- Set  $\text{left} = 0$ .
- Set  $\text{right} = n - 1$ .
- Find the middle position using:  
 $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$ .
- Compare  $\text{arr}[\text{mid}]$  with the target.
- If  $\text{arr}[\text{mid}] == \text{target}$ , return the index.
- If  $\text{arr}[\text{mid}] < \text{target}$ , search in the right half.
- If  $\text{arr}[\text{mid}] > \text{target}$ , search in the left half.
- Repeat until the target is found or  $\text{left} > \text{right}$ .
- If the target is not found, return -1.
- Stop.

Note: The array must be sorted before applying Binary Search.

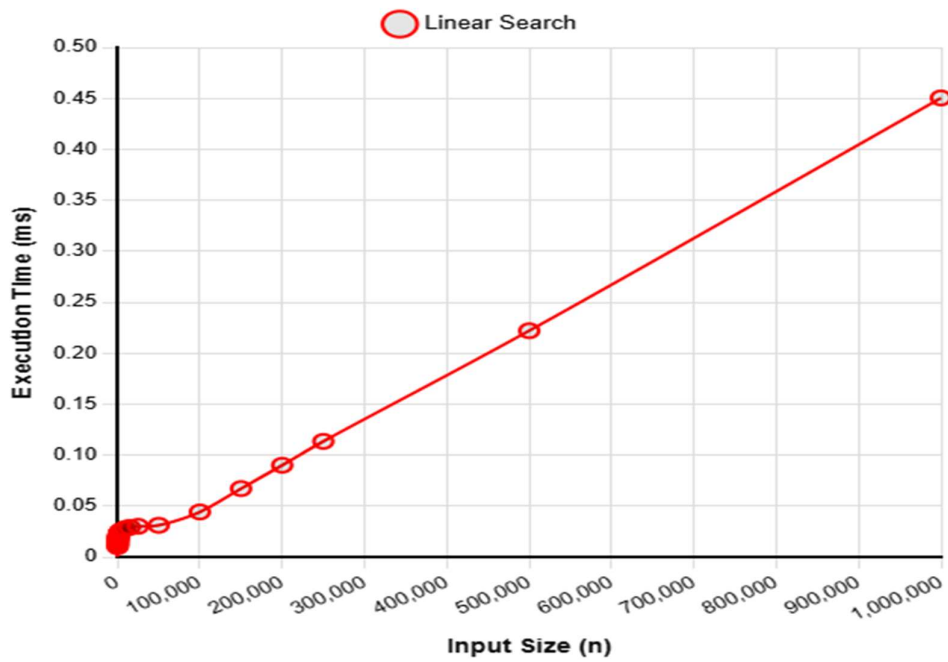
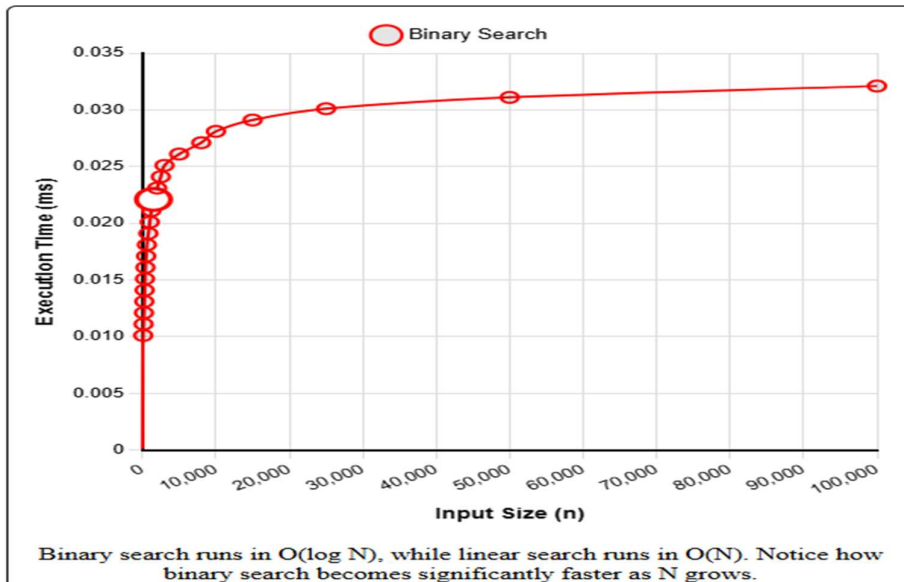
**Pseudocode:** BinarySearch(arr, target):

```
left = 0
right = n - 1
while left <= right:
    mid = left + (right - left) // 2
    if arr[mid] == target:
        return mid
    else if arr[mid] < target:
        left = mid + 1
    else:
```

right = mid - 1

return -1

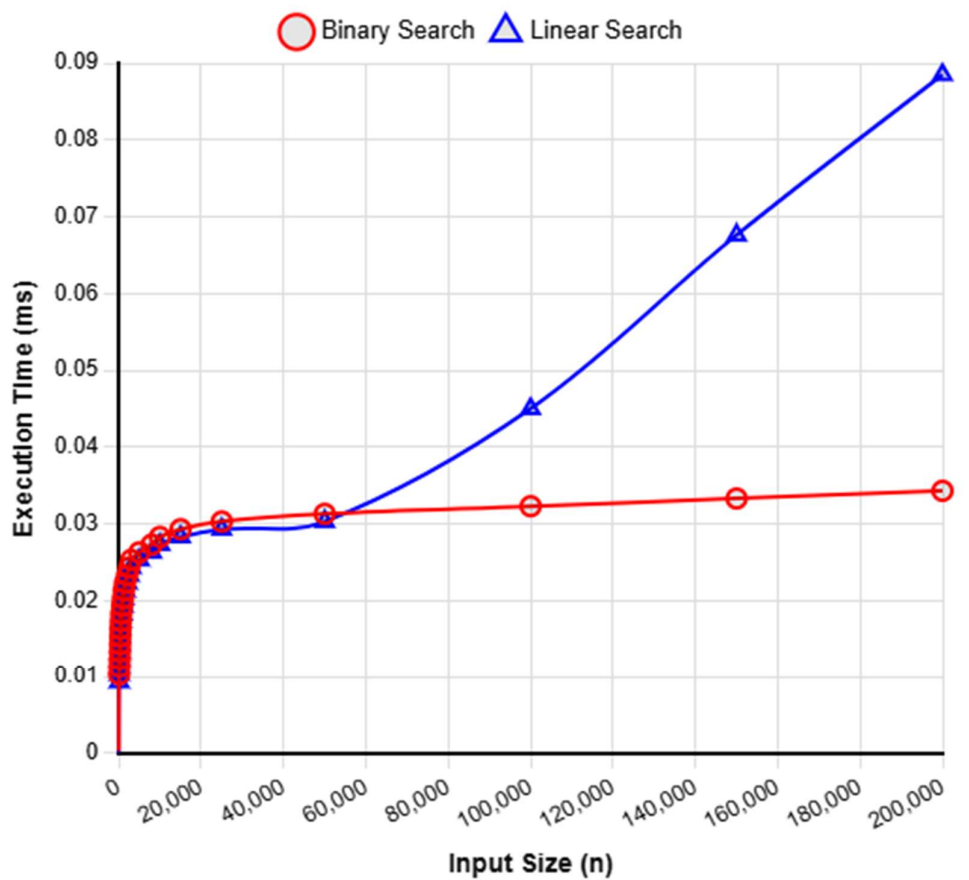
## OUTPUT:



## TIME AND SPACE COMPLEXITY:

Binary Search:  $O(\log N)$

Linear Search:  $O(N)$



Binary search runs in  $O(\log N)$ , while linear search runs in  $O(N)$ . Notice how binary search becomes significantly faster as  $N$  grows.